

Delegates Real-Time Example in C#

In this article, I am going to discuss **Delegates in C# with Real-Time** Examples. The delegates are one of the most important concepts that you need to understand as a C# developer. In many interviews, most of the interviewers ask you to explain the usage of delegates in the real-time project that you have worked on. Please read the following two articles, where we discussed the basics of delegate and multicast delegates in C# with examples.

1. [Single Cast Delegates in C#](#) – Here we discussed the basics of Delegates in C# with examples.
2. [Multicast Delegate in C#](#) – Here we discussed the Multicast Delegates in C# with examples.

The Delegates in C# are extensively used by framework developers. Let us understand delegates in C# with two real-time examples.

Delegate Real-Time Example1 in C#:

Let's say we have a class called Worker and this class has a method called DoWork. What our business requirement is when we invoke the DoWork method, we need to send a notification about the percentage of work done to the consumer as well as once the work is completed, we also need to send the notification. For example, how much work is done in 1 hour, how much work is done in 2 hours, how much work is done in 3 hours, and so on until the work is completed? And the DoWork method has no idea to whom to send the notifications. The caller of the DoWork method should decide to whom to send the notification. That means here we need to use call-back functions.

This is an ideal scenario where we need to use delegates. So, create a class file with the name Worker.cs and then copy and paste the following code into it. Here, you can see that we have created two delegates. You can create delegates anywhere and you can also create delegates within the class or directly inside the namespace. This is because behind the scene the delegates are classes only. The DoWork method takes four parameters, out of the four, two parameters are of the delegate type. Then within the method, we are using a for loop and each time the loop executes, we are processing some work and notifying the user by invoking the delegate. Once the loop execution is completed, which means our work is done, and then we are invoking the work completed delegate to notify the user that the work has been done.

```
using System.Threading;
namespace DelegatesRealTimeExample
{
    public delegate void WorkPerformedHandler(int hours, string workType);
    public delegate void WorkCompletedHandler(string workType);
    public class Worker
    {
        public void DoWork(int hours, string workType, WorkPerformedHandler del1,
        WorkCompletedHandler del2)
        {
            //Do Work here and notify the consumer that work has been performed
            for (int i = 0; i < hours; i++)
            {
                //Do Some Processing
                Thread.Sleep(1000);
                //Notfiy how much works have been done
                del1(i+1, workType);
            }
        }
    }
}
```

```

    }
    //Notfiy the consumer that work has been completed
    del2(workType);
}
}
}

```

Now, modify the Program class as follows. Here, we are creating two call-back methods with the same signature as our delegates. Then we are creating instances of delegate and also we are creating an instance of the Worker class and invoking the DoWork method by passing the required values along with the two delegate instances.

```

using System;
namespace DelegatesRealTimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            WorkPerformedHandler del1 = new WorkPerformedHandler(Worker_WorkPe
            WorkCompletedHandler del2 = new WorkCompletedHandler(Worker_WorkCc
            Worker worker = new Worker();
            worker.DoWork(5, "Generating Report", del1, del2);
            Console.ReadKey();
        }
        //Delegate Signature must match with the method signature
        static void Worker_WorkPerformed(int hours, string workType)
        {
            Console.WriteLine($"{hours} Hours compelted for {workType}");
        }
        static void Worker_WorkCompleted(string workType)
        {
            Console.WriteLine($"{workType} work compelted");
        }
    }
}

```

Now, run the application and you will see that each time the loop is executed, it will send the notification and once the work is done, then it will send another notification as shown in the below image.

```

1 Hours compelted for Generating Report
2 Hours compelted for Generating Report
3 Hours compelted for Generating Report
4 Hours compelted for Generating Report
5 Hours compelted for Generating Report
Generating Report work compelted

```

That is working fine. Now, if you go to the Worker class and within the DoWork method, then it allows you to make the delegate instance null.

```

using System.Threading;
namespace DelegatesRealTimeExample
{
    public delegate void WorkPerformedHandler(int hours, string workType);
    public delegate void WorkCompletedHandler(string workTyp);
    public class Worker
    {
        public void DoWork(int hours, string workType, WorkPerformedHandler del1,
        WorkCompletedHandler del2)
        {
            del1 = null; //Allowing to set null
            del2 = null; //Allowing to set null
            //Do Work here and notify the consumer that work has been performed
            for (int i = 0; i < hours; i++)
            {
                //Do Some Processing
                Thread.Sleep(1000);
                //Notfiy how much works have been done
                del1(i + 1, workType);
            }
            //Notfiy the consumer that work has been completed
            del2(workType);
        }
    }
}

```

So, here, if the Worker class not creating the instance and if the Worker class has no idea about the methods to whom he sends notifications, then it should not allow making the delegate instance null. How we can restrict this? By using [Events in C#](#) and which we will discuss in our next article.

Delegates Real-Time Example2 in C#:

Now, we will see another real-time example of delegates in C#. Let us say we have a class called Employee as shown below.

```

namespace DelegateRealtimeExample
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public int Experience { get; set; }
        public int Salary { get; set; }
    }
}

```

The above Employee class has the following properties.

1. Id

2. **Name**
3. **Gender**
4. **Experience**
5. **Salary**

Now I want to write a method in the Employee class which can be used to promote the employees. The method that we are going to write will take a list of Employee objects as a parameter and then should print the names of all those employees who are eligible for a promotion.

But the logic based on which the employee gets promoted should not be hard coded. At times we may promote employees based on their experience and at times we may promote the employees based on their salary or maybe based on some other reason like the performance of the employee. So, the logic to promote the employees should not be hard-coded within the method.

How we can achieve this? This can be achieved very easily with the help of delegates. So, now I need to design my Employee class as shown below.

```
using System;
using System.Collections.Generic;
namespace DelegateRealtimeExample
{
    public delegate bool EligibleToPromotion(Employee EmployeeToPromotion);
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public int Experience { get; set; }
        public int Salary { get; set; }
        public static void PromoteEmployee(List<Employee> lstEmployees,
        EligibleToPromotion IsEmployeeEligible)
        {
            foreach (Employee employee in lstEmployees)
            {
                if (IsEmployeeEligible(employee))
                {
                    Console.WriteLine($"Employee {employee.Name} Promoted");
                }
            }
        }
    }
}
```

In the above code, we have created a delegate called EligibleToPromotion. This delegate takes the Employee object as a parameter and returns a boolean value indicating whether the employee is promoted or not. In the Employee class, we also created the PromoteEmployee method and this method takes a list of Employees and a Delegate of the type EligibleToPromotion as parameters.

Then, the PromoteEmployee method loops through each employee's object and passes it to the delegate. If the delegate returns true, then the Employee is promoted, else not promoted. So, within the method, we have not hardcoded any logic on how we want to promote employees.

Now the client who uses the Employee class has the flexibility of determining the logic of how they want to promote their employees. First create the employee objects – emp1, emp2, and emp3. Populate the properties for the respective objects. Then create an employee List to hold all the 3 employees as shown below.

```
using System;
using System.Collections.Generic;
namespace DelegateRealtimeExample
{
    public class Program
    {
        static void Main()
        {
            Employee emp1 = new Employee()
            {
                ID = 101,
                Name = "Pranaya",
                Gender = "Male",
                Experience = 5,
                Salary = 10000
            };
            Employee emp2 = new Employee()
            {
                ID = 102,
                Name = "Priyanka",
                Gender = "Female",
                Experience = 10,
                Salary = 20000
            };
            Employee emp3 = new Employee()
            {
                ID = 103,
                Name = "Anurag",
                Experience = 15,
                Salary = 30000
            };
            List<Employee> lstEmployess = new List<Employee>();
            lstEmployess.Add(emp1);
            lstEmployess.Add(emp2);
            lstEmployess.Add(emp3);
            EligibleToPromotion eligibleTopromote = new
            EligibleToPromotion(Program.Promote);
            Employee.PromoteEmployee(lstEmployess, eligibleTopromote);
            Console.ReadKey();
        }
        public static bool Promote(Employee employee)
        {
            if (employee.Salary > 10000)
            {
                return true;
            }
        }
    }
}
```

```

        else
        {
            return false;
        }
    }
}

```

Notice the Promote method that we have created. This method has the logic of how we want to promote our employees. The method is then passed as a parameter to the delegate. Also, note this method has the same signature as that of the EligibleToPromotion delegate. This is very important because the Promote method cannot be passed as a parameter to the delegate if the signature differs. This is the reason why delegates are called type-safe function pointers. Now, run the application and you will see the following output.

```

Employee Priyanka Promoted
Employee Anurag Promoted

```

So, if we did not have the concept of delegates, it would not have been possible to pass a function as a parameter. As Promote method in the Employee class makes use of delegates, it is possible to dynamically decide the logic of how we want to promote employees.

In C# 3.0 Lambda expressions are introduced. So, you can make use of lambda expressions instead of creating a function and then an instance of a delegate and then passing the function as a parameter to the delegate. The following example is rewritten using the Lambda expression. The Private Promote method is no longer required now.

```

using System;
using System.Collections.Generic;
namespace DelegateRealtimeExample
{
    public class Program
    {
        static void Main()
        {
            Employee emp1 = new Employee()
            {
                ID = 101,
                Name = "Pranaya",
                Gender = "Male",
                Experience = 5,
                Salary = 10000
            };
            Employee emp2 = new Employee()
            {
                ID = 102,
                Name = "Priyanka",
                Gender = "Female",
                Experience = 10,
                Salary = 20000
            };
            Employee emp3 = new Employee()

```

```

    {
        ID = 103,
        Name = "Anurag",
        Experience = 15,
        Salary = 30000
    };
    List<Employee> lstEmployess = new List<Employee>();
    lstEmployess.Add(emp1);
    lstEmployess.Add(emp2);
    lstEmployess.Add(emp3);
    Employee.PromoteEmployee(lstEmployess, x => x.Experience > 5);
    Console.ReadKey();
}
}
}

```

Output:

```

Employee Priyanka Promoted
Employee Anurag Promoted

```

In the next article, I am going to discuss [Generic Delegates in C#](#) with Examples. Here, in this article, I try to explain **Delegates Real-Time Examples in C#**. I hope you enjoy this C# Delegate Real-Time Examples article.